Agentic System Design Process

Nine phases from problem to production, assembled from the course materials and practitioner guides

J. A. Arroyo

Version 1.8 — 10 October 2026 (Phase 2 wording aligned with the screening companion)

Abstract

I did not find a complete lifecycle process in the public materials reviewed from the fifteen courses indexed in the guide [26]. Individual courses cover some phases explicitly, through project templates, rubrics and slides, and the practitioner guides from Anthropic, OpenAI and Google cover most of the rest. This document combines those sourced practices with my synthesis. **The phase order, the gates and the resolutions between sources are inferred recommendations; the citations identify support for individual practices.** Places where the sources differ are stated, not hidden. [inferred] marks a statement that the cited sources do not explicitly prescribe.

Contents

How to use this document 2

Phase 1: Frame the problem 2

Phase 2: Decide whether an agent is warranted 3

Phase 3: Decompose the task and assign ownership 5

Phase 4: Define the evaluation 6

Phase 5: Design tools, state and context 7

Phase 6: Build a minimal prototype and select models 7

Phase 7: Evaluate, diagnose and iterate 8

Phase 8: Harden safety and approve release 9

Phase 9: Deploy gradually and maintain 10

Different emphases and my design choices 10

Limits of this document 10

Authorship and use of AI tools 11

Sources 11

How to use this document

One set of three documents

1. **Learning to Build Agentic Systems** (the guide): the courses, lectures and assignments behind the practices. *Use it to learn a topic.*
2. **Agentic System Design Process**: nine phases from framing a problem to running it in production. *Use it to design a system.*
3. **Screening Tasks for Agentic Automation**: a one-page worksheet, filled in after Phase 1 of the design process, that feeds Phase 2 and sets up the comparison run in Phase 7. *Use it to decide whether a task is worth it.*

This is document 2. The courses it cites are indexed in the guide; the worksheet it uses in Phase 2 is in the screening companion [27].

Work the phases in order, and write down the output of each phase before moving on. The process is not linear in practice:

- Between Phase 1 and Phase 2, fill in the screening worksheet [27, Section 6]. Its answers are the input to Phase 2, and its last row is the comparison that Phase 7 runs.
- Phase 7 can send you back to Phase 2 (simplify) or Phase 3 (re-split the work).
- Phase 9 triages production failures and adds representative regression cases to the Phase 4 set.
- Safety is not a late add-on. It is designed in Phases 1, 3 and 5 and tested in Phase 8.

Scale each phase to the stakes. For a small internal tool, a phase can be a paragraph; for a customer-facing system, it is a reviewed document.

#	Phase	Output	Support
1	Frame the problem	Problem brief with success and unacceptable failures	Strong
2	Decide whether an agent is warranted	Option comparison; written justification	Strong (guides)
3	Decompose and assign ownership	Task map: code / model / human; stop and approval points	Strong
4	Define the evaluation	Test set, metrics, baseline, release bar	Very strong
5	Design tools, state and context	Tool contracts, state model, permission boundaries	Strong
6	Prototype and select models	One traced working path; model comparison	Practices strong; order disputed
7	Evaluate, diagnose, iterate	Error analysis; improvements vs. baseline	Very strong
8	Harden safety and approve release	Adversarial results; release sign-off	Strong
9	Deploy gradually and maintain	Versioned rollout, monitoring, regression growth	Strong (guides); weak (courses)

Figure 1 shows the phases, the return paths and where safety work happens.

Phase 1: Frame the problem

State what problem is being solved, for whom, and how you will know it is solved, before any design choice is made.

Questions to answer

- Who is the user, and what task are they trying to complete?
- How is it done today, and what is wrong with that? Why does it matter?
- What data or inputs are available, and what constraints apply (cost, latency, privacy, regulation, deployment environment)?
- What observable result counts as success?

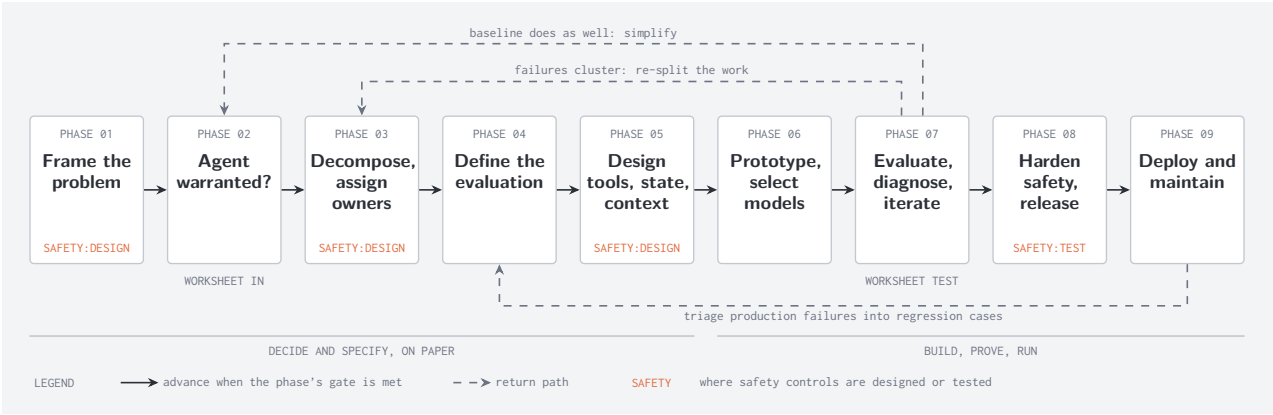


Figure 1: The nine phases. Return paths and safety touchpoints follow the text; the order and the gates are my synthesis [inferred]. The screening worksheet [27] is filled in before Phase 2, and its comparison is run in Phase 7.

- Which failures are unacceptable (wrong irreversible action, data leak, confident wrong answer)? These seed the safety work in Phases 3, 5 and 8.

Evidence. Colorado's project proposal asks for the problem, why it matters, the state of the art and its limits, the proposed solution, and how you would convince a reader it works [29]. Mannheim's project outline requires four answers: the problem, the data, the method including which LLMs, and how success is measured [28, slide 18]. Buffalo's final project begins with a problem statement and an environment and tool inventory [23]. OpenAI and Google both start from the use case and its business problem [5, pp. 5–6][6, pp. 19–20].

Done when

A one-page brief exists that names the user, the task, the constraints, the success measure and the unacceptable failures.

Phase 2: Decide whether an agent is warranted

Choose the simplest option that can meet the Phase 1 success measure, and justify in writing every step up in complexity.

Input. The Phase 1 brief and the completed screening worksheet [27, Section 6]. Row S records how work starts. Rows 0a and 0b test the simpler alternatives; rows A1 to A3 assess further model-assisted automation when those alternatives are not enough; rows B1 to B4 identify the control decisions left unresolved and plan the comparison.

Before the ladder: alternatives without a language model. First ask whether a conventional statistical or machine-learning model, or a numerical optimizer, would meet the success measure, alone or alongside a script. Record the strongest such option and the evidence that it meets or misses the target; if that is unknown, run the smallest test that would tell you. These are not a rung of the ladder, because they can also serve as tools at any rung [inferred]. Mannheim's non-LLM baseline is the closest course precedent [28, slide 26]; the screening companion gives the cases behind this check [27, Section 3].

The ladder (Figure 2). Consider the options in this order and stop at the first one that is adequate [inferred]:

1. **Deterministic code** (rules, scripts, a form). Predictable, cheap, testable.
2. **A single LLM call**, optionally with retrieval and examples.
3. **A fixed workflow**: several LLM calls and code steps in a path set in advance (chaining, routing, parallelisation).
4. **A bounded agent**: the model chooses the next step in a loop, with tools, limits and stopping rules.

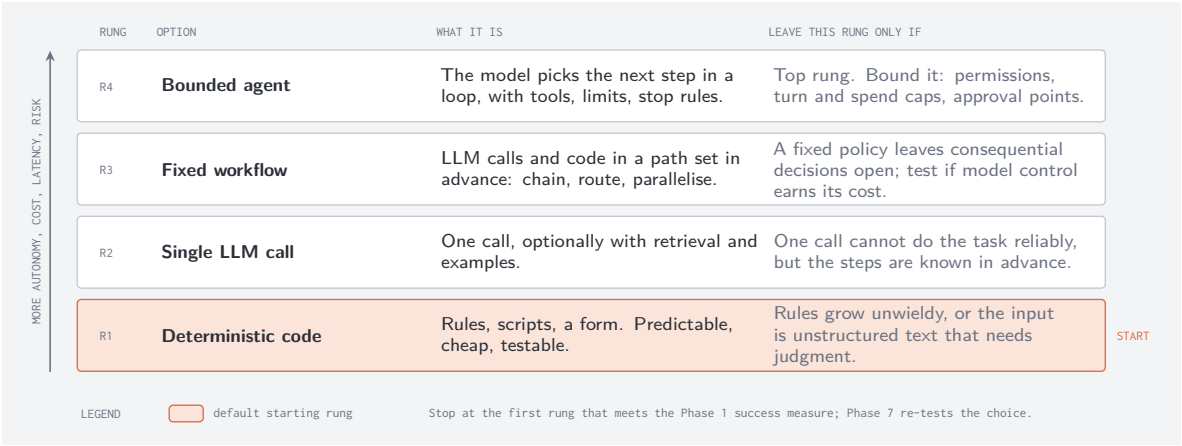


Figure 2: The Phase 2 ladder. The exit conditions paraphrase Anthropic [1, "When (and when not) to use agents"], OpenAI [5, pp. 5–6] and Northeastern [20, slides 9, 10, 14]; arranging them as a ladder, and the tighter wording of the exit from the fixed workflow, are my synthesis [inferred].

Criteria for moving up.

- Anthropic: find the simplest solution possible and add complexity only when it demonstrably improves results; workflows suit well-defined tasks, agents suit open-ended problems where the steps cannot be predicted, at the price of cost, latency and compounding errors [1, "When (and when not) to use agents"].
- OpenAI: consider an agent when the task needs complex judgment, the rule set has become unwieldy, or it relies heavily on unstructured data; otherwise a deterministic solution may suffice [5, pp. 5–6].
- Northeastern distinguishes fixed workflows and one-time routing from a loop in which the model selects actions using their results. Cycles alone are insufficient: the model must direct the next action [20, slides 9, 10, 14].
- CMU places developer-defined workflows, plan-then-execute and adaptive agents on one spectrum: a fixed plan is faster, cheaper and more predictable, and replanning should be bought only when its feedback is worth the cost [15, p. 16].
- Stanford CS146S, for skills: a single fixed command is a shell alias, not a skill; a skill needs repetition and a real decision point [18, Part I].

Leaving the fixed workflow. "The steps cannot be predicted" is not enough to leave the fixed-workflow rung, because a fixed workflow can branch, retry and escalate. Write the strongest fixed policy you can maintain, including those branches, and list the decisions it leaves unresolved. No branch may hide an agent: "diagnose and repair" is not a step unless the policy says how. Move up only if those unresolved decisions are worth more than the added cost and risk [inferred].

Before authorizing a bounded agent, answer three questions about feedback [inferred]:

- What observation will guide the model's next decision during a run?
- What checks success independently of the model, in a way the agent cannot alter?
- How has that check been tested, for example on cases already known to be good and bad?

Weak or misleading feedback can misdirect the agent's adaptation. Record its limitations and test whether it is informative enough for the decisions the agent will make [inferred]. The screening companion gives the cases behind these feedback questions and the exit from the fixed workflow [27, Sections 3–4].

Gap in the courses. In the public material checked, no course asks students to write down why a script or a single call would not do. Only the Anthropic and OpenAI guides state the test. This phase makes it an explicit gate. The decision is provisional: Phase 7 confirms it empirically by comparing against the simpler option, as

CS329A HW3 does when it builds a single-call baseline before adding routing, decomposition and refinement [32, Parts 0–3], and as Mannheim does with a non-LLM baseline [28, slide 26].

Done when

The chosen option is recorded, with the evidence for rejecting each relevant simpler alternative and any remaining unknowns. If the provisional choice is a bounded agent, the record also holds the strongest maintainable fixed policy, the decisions it appears to leave unresolved, their expected value, the answers to the three feedback questions and the planned comparison.

Phase 3: Decompose the task and assign ownership

Split the work into tasks and subtasks, and decide for each whether code, the model or a human owns it.

Allocation rules (Northeastern, week 3b, rules 1–6 [19]; Figure 3):

- Policy, calculations, permissions and budgets live in **code**; contextual judgment and tool selection live in the **model**. "The model proposes; the runtime disposes."
- Prompt rules that read like code belong in code.
- Tools validate their own inputs, obtain their own prerequisites, and return both facts and verdicts, so they cannot be talked out of a rule.

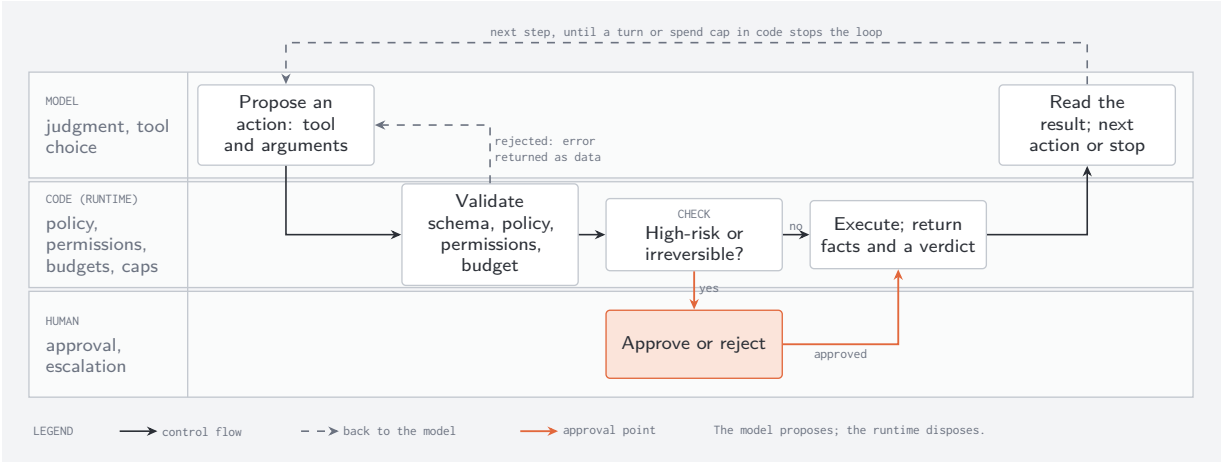


Figure 3: Ownership for one step of an agent loop: the model proposes, code validates and executes, and a human approves high-risk actions. Allocation follows Northeastern [19, rules 1–6]; human intervention on high-risk actions follows OpenAI [5, pp. 24–31]. The sequence is an illustration [inferred].

Design checklist (CMU lecture 5, p. 46 [15]):

- Are the subtasks really separable, and would different parts benefit from different models?
- Which observations should change the plan?
- How will errors and context be kept from piling up?
- What information or authority must stay separate, and who approves irreversible actions?

Also record: stopping conditions (turn and spend caps), the points where a human must approve, and the escalation path when the agent is unsure. LangGraph's method maps the workflow into discrete steps and labels each as an LLM step, a data step, an action or a human-input step [9, steps 1–2]. OpenAI requires human intervention on failure thresholds and high-risk actions [5, pp. 24–31].

Single or multiple agents. OpenAI recommends maximising a single agent first and considering a split when complex instructions or overlapping tools impair performance [5, p. 16]. Anthropic describes additional patterns for parallel work and dynamic delegation [1, "Parallelization", "Orchestrator-workers"]. Berkeley's Lab 2 recommends planning, generation and verification agents for its Lean task [37]; treat that as a design for that task, not a default.

Done when

A task map lists every subtask with its owner (code, model, human), the stopping conditions, and the approval points.

Phase 4: Define the evaluation

Decide how success will be measured before building, so the prototype is judged against evidence rather than demos.

Contents

- **A first test set.** Anthropic recommends starting early with 20–50 simple tasks drawn from real failures, because evals get harder to build the longer you wait [3]. Include known-good cases, known failures and edge cases. Berkeley requires test cases with known outcomes, including edge cases, and asks that a human be able to solve each task [35, pp. 82–85, 91]. Those known-outcome cases test the evaluator itself [35, p. 85].
- **Metrics** across quality, cost, latency and safety. Mannheim's method: define success criteria, choose the method (code checks, LLM judge, human review), use tooling, iterate [24, p. 76]. CMU evaluates at four levels: tool selection, arguments, trajectory and final task, plus efficiency, reliability and safety [14, pp. 54–56].
- **Reliability, not only success rate.** Run each task several times to measure consistency; test robustness by injecting tool faults, changing tool-interface formats and paraphrasing prompts; and, if the agent reports confidence, measure its calibration. Rabanser et al. define such metrics in four groups (consistency, robustness, predictability, safety) and used five runs per task, a fault probability of 0.2 and five paraphrases [13, §3, Table 2; §4.1]. Five runs is their experimental setting, not a prescribed minimum; choose the count to balance estimation precision against cost [inferred].
- **The grader.** Prefer code checks, then model graders, with humans to calibrate [3]. An LLM judge is not always needed: verifiable tasks can be checked objectively [35, pp. 17, 71].
- **Development and held-out cases.** Use development cases for iteration and model selection, and keep held-out cases to check generalisation before release. Validate graders against known successes and failures, and reset environment and memory state between independent trials [2, "Collaborating with agents"][3, steps 2, 4, 5].
- **The baseline**: the simpler rung from Phase 2, run on the same set.
- **The release bar**: the threshold, on these metrics, that Phase 8 must confirm. For reliability, consider pass^k (success on all k attempts) rather than pass@k [36, p. 30].

Order dispute. The Anthropic evals post and the Berkeley project put evaluation first (Berkeley's project structure introduces evaluator development as Phase 1 and participant-agent development as Phase 2 [38, Grading, Project Timeline]; both final submissions share a deadline, so this is an emphasis, not a strict gate); the Anthropic tools post prototypes first and then evaluates [2]; OpenAI sets up evals as the baseline for model selection [5, p. 8]. Resolution [inferred]: write a small set before building, then grow it from the failures found in Phase 7.

Done when

A versioned test set, the metrics, the grader for each, the baseline and the release bar are written down.

Phase 5: Design tools, state and context

Specify everything the model can touch and see, so that its actions are constrained by design rather than by prompt wording.

Tools

- Prefer a few workflow-level tools to many thin endpoint wrappers; namespace them; return high-signal, token-efficient results; make errors actionable [2].
- Put constraints in the schema (enums, ranges, required fields), shape the outputs, return errors as data, treat the docstring as the contract, and put a brake on write tools (read-only hints, dry runs) [17, Part II].
- Design the interface to be hard to misuse (poka-yoke), for example requiring absolute paths [1, Appendix 2].
- Call a tool only when its benefit outweighs its latency, cost, failure modes and risk [14, p. 5].

State and context

- Define the state the system carries between steps, how it changes, and what happens on each error path [9, steps 3–4].
- Decide what context each step needs and keep it minimal and high-signal [4]. Persistent memory is a design choice, not a required component.
- For a recurring agent, track progress per source with a stored bookmark rather than a fixed time window. If a source cannot be read, keep its bookmark and say so in the output; never report it as "nothing new" [11, Sources].
- Record a side effect as done only after the destination confirms it, and check for an existing result before retrying to reduce duplicate effects [11, Destination].
- Keep the rules the agent must follow in a store it can read but not edit; have it re-read them every run and stop if it cannot [11, Memory].
- Treat curated context as an owned asset. Notion separates data context (metrics, sources, required filters), business context and routing guidance, and assigns owners and review to keep them current [12, "Connecting data and business context", "Keeping data and business context current"].

Permission boundaries. Least privilege for every tool and credential, sandboxed execution, and paths confined to a working directory [30][16]. Minimal OAuth scopes, each with a stated reason [17]. When the agent acts for a user, let it use that user's own credentials so existing access controls apply, and have it report denied access instead of working around it [12, "Governing the agent and its data access", "Clear instructions for repeatable analysis"].

Done when

Every tool has a contract (inputs, outputs, errors, side effects, permission level); the state model and error paths are written; permissions are listed.

Phase 6: Build a minimal prototype and select models

Get one path working end to end with full tracing, then choose models by measuring them on the Phase 4 development set.

Build

- Build the simplest version of the chosen rung. Anthropic suggests starting with direct API calls rather than a framework, so the mechanics stay visible [1]; Google calls a production-grade framework essential [6, p. 23]. Resolution [inferred]: specify the capabilities you need and choose a library only if it supplies them.

- Test each tool on its own before wiring it in, and write the system prompt last [21].
- Trace every run: prompts, tool calls, results, cost and latency. Phase 7 depends on it.

Select models

- OpenAI: prototype with the most capable model to set a quality baseline, then try smaller models and keep them where they still meet the target [5, p. 8].
- Mannheim recommends free-model prototyping, small-model main experiments, and selective use of stronger models after estimating cost [28, slide 28]. CS329A recommends smaller, cost-effective models during development before scaling to larger models [32, "API Configuration and Setup"].
- JHU A2 compares models empirically across languages and prompting strategies [31].
- Resolution [inferred]: these optimise different budgets. If quality is uncertain, start strong to learn what is achievable; if spend is the binding constraint, start cheap and escalate. In both cases compare on the Phase 4 development set, and record quality, cost and latency per model.

Done when

One traced path runs end to end, and a model table (quality, cost, latency on the development set) supports the chosen model or models.

Phase 7: Evaluate, diagnose and iterate

Find out why the system fails, fix the causes, and confirm that the added complexity earns its cost.

The loop (CS329T HW1, the course's "standard iterative process" [33, §§1–2]; Figure 4):

1. Run the system on the development set and read the traces.
2. Label failures by hand, grouped into error classes.
3. If you use LLM judges, check them against your hand labels before trusting them.
4. Change prompts, tools or architecture and rerun.

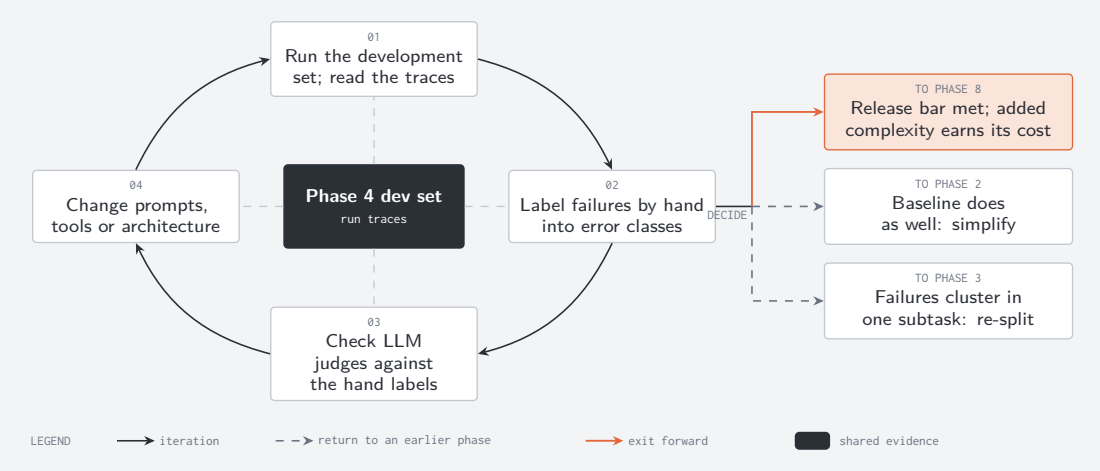


Figure 4: The Phase 7 loop. Steps 01–04 follow CS329T HW1 [33, §§1–2] and Mannheim [28, slide 26]; the exits and their conditions are my synthesis [inferred].

Supporting practice

- Mannheim: define error classes by hand on a sample, count them, then automate the analysis [28, slide 26].

- CS329T's Agent GPA checks goal fulfilment, plan quality, plan adherence, tool selection, tool calling, execution efficiency and logical consistency [34, slide 21].
- Google: evaluate end-to-end outcomes first, then inspect trajectories to diagnose [7, pp. 17–19]. Chip Huyen catalogues planning and tool failure modes and recommends testing tools independently [10].
- Separate capability evals (can it do this?) from regression evals (did a change break it?) [3].
- A run without errors does not establish that its answer is correct. Notion found that a query could run cleanly and still answer the wrong question, with the meaning, routing, measurement or interpretation wrong, and that more retrieval or retries did not automatically help [12, "Why working SQL isn't enough", "Seeing what the agent actually did"].

When Phase 2 chose a bounded agent, the baseline is the strongest fixed workflow, and the comparison is the matched one planned on the screening worksheet: the same model, starting information, tools, check and limits in both arms, on cases used to design neither, so that the only difference is who selects the next step [27, Section 4].

Return paths. If the system does not beat the Phase 2 baseline by enough to justify its cost and latency, go back to Phase 2 and simplify. If failures cluster in one subtask, revisit its ownership in Phase 3 [inferred].

Done when

Error classes and counts are recorded, graders are validated, and the selected design meets the agreed success and safety thresholds. Any added complexity has a measured benefit that justifies its cost and latency; otherwise simplify and re-evaluate [inferred].

Phase 8: Harden safety and approve release

Show that the boundaries from Phases 3 and 5 hold under attack and misuse, and that the release bar is met.

- Run adversarial and misuse cases: prompt injection through tool outputs and retrieved content, attempts to exceed permissions, and inputs that should be refused. Buffalo's Lab 5 asks where autonomy helps and where it should be restricted [23]; CMU covers sandboxing and approval points [16].
- Layer guardrails: LLM-based checks, rules and moderation, added as edge cases appear; route high-risk actions and repeated failures to a human [5, pp. 24–31]. Guardrails can run as separate parallel calls [1].
- Guardrails can miss attacks; Mannheim's safety deck covers their limits [25, pp. 17–23]. Measure their effectiveness against the system's threat model, rely on enforced permission boundaries, and explicitly assess residual risk [inferred].
- Production concerns listed by Sierra: guardrails and enforced determinism, regression testing, user simulation, staging and release, model migration and failover [36, p. 10].
- Treat safety results as hard constraints: report them separately instead of averaging them into an overall score [13, §3.5]. Where the agent's output is the final action, with no human review, set a higher reliability bar than for an assistant [13, §5, Recommendation 4].
- For agents that run unattended, use read-only credentials wherever the agent only reads, and cap what each run can spend. Anthropic suggests starting the cap at three to five times the cost of a normal run and tightening it from real numbers [11, Guardrails].
- Confirm the Phase 4 release bar, including the held-out generalisation check and regression and safety evaluations, and record who signed off. Google describes evaluation gates before release, including a manual gate for less mature teams [8, pp. 12, 17–18].

Done when

Adversarial results are recorded, every unacceptable failure from Phase 1 has a tested control, and the release bar is met and signed off.

Phase 9: Deploy gradually and maintain

Release in stages with a way back, watch the system in use, and turn what you learn into tests.

- Version prompts, tools, models and the test set together. Roll out in stages with a rollback path [8, pp. 15–16, 24].
- Monitor quality, cost, latency and safety events in production. Monitoring complements evals; it does not replace them [3].
- Triage production failures and add representative regression cases to the Phase 4 set [8, p. 24].
- Re-run the full evaluation on any model change [8, p. 18]. Sierra's development cycle is build, test, optimise, repeated [36, p. 17].
- Re-evaluate at regular intervals, not only after changes. Rabanser et al. call for periodic evaluation on evolving benchmarks, so that reliability that degrades silently is caught [13, §5 and App. A, Recommendation 1]. In deployment, apply this by regularly re-evaluating against cases and environments updated to reflect current conditions [inferred].
- Start small and validate with real users [5, conclusion]. For a scheduled agent, trigger a run by hand before relying on the schedule [11, Schedule].
- Review production traces on a schedule. At Notion, a nightly review agent records missing, stale or conflicting context, and a separate triage agent applies authorised fixes and sends ambiguous or higher-risk changes to a person; it notes that query success and rising usage are useful signals, but "neither proves accuracy" [12, "Keeping data and business context current", "Seeing what the agent actually did"].

Done when

The system is versioned, the rollout and rollback plan is tested, monitoring is live, and there is a routine for moving production failures into the test set.

Different emphases and my design choices

Question	Positions	How this process treats it
Model order	Strongest first, then downsize [5]; cheap first, then escalate [28, 32]	Choose by budget; always compare on the development set (Phase 6)
Evals or prototype first	Evals early [3]; evaluator-first project structure [38, Grading, Project Timeline]; prototype then evaluate [2]	Small set before building, grown from failures (Phase 4)
One agent or several	Single agent first [5, p. 16]; three agents for a Lean task [37]	Start simple; add agents when measured benefits justify the coordination cost [inferred] (Phase 3)
Frameworks	Direct API calls first [1]; framework essential [6]	Specify capabilities, not a library (Phase 6)
LLM judges	Common in course rubrics [33]; objective checks for verifiable tasks [35]	Code checks first; validate judges against humans (Phases 4, 7)
Research framing	Mechanism → manipulation → measured behaviour [22]	Research novelty is not a product requirement; mechanism-based experiments can still test design choices [inferred]

Limits of this document

- The evidence is the public material checked on 9 October 2026. Unpublished rubrics (CMU's project, Buffalo, Stanford CS146S's final project), JHU lecture PDFs over 10 MB, Google Slides and PPTX decks, and lecture videos were not read. "Not found" means not found in that material.
- The Anthropic, OpenAI and Google guides were read through extracted text and summaries; page and section numbers come from those reads. The reliability paper [13] was read in its arXiv HTML version (v3). The two practitioner posts [11, 12] each describe one system; Notion's outcome figures are self-reported.

- The phase order, the Phase 2 ladder, the gates and the resolutions of disagreements are my synthesis.

Authorship and use of AI tools

I am the sole author. I used AI tools in preparing this document and disclose their role here. *Claude* (Anthropic, through Claude Code) read the course materials and practitioner guides, and drafted and revised the text and diagrams under my direction. *Codex* (OpenAI) made an independent pass over the same sources and reviewed the citations; its corrections were applied. Words taken from a source are in quotation marks with the source. Practices taken from a source are cited to it, with a page or section reference where one is available. Statements marked [inferred] are my own synthesis. I set the questions, decided what to include and am responsible for the content.

References

[1] Anthropic, "Building Effective Agents". https://www.anthropic.com/engineering/building-effective-agents

[2] Anthropic, "Writing effective tools for agents". https://www.anthropic.com/engineering/writing-tools-for-agents

[3] Anthropic, "Demystifying evals for AI agents" (Jan 2026). https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

[4] Anthropic, "Effective context engineering for AI agents". https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[5] OpenAI, "A practical guide to building agents". https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

[6] Google/Kaggle, "Introduction to Agents" (May 2026). https://www.kaggle.com/whitepaper-introduction-to-agents

[7] Google/Kaggle, "Agent Quality" (May 2026). https://www.kaggle.com/whitepaper-agent-quality

[8] Google/Kaggle, "Prototype to Production" (May 2026). https://www.kaggle.com/whitepaper-prototype-to-production

[9] LangChain, "Thinking in LangGraph". https://docs.langchain.com/oss/python/langgraph/thinking-in-langgraph

[10] Chip Huyen, "Agents" (Jan 2025), section "Agent Failure Modes and Evaluation". https://huyenchip.com/2025/01/07/agents.html

[11] L. Martin and C. Avilla (Anthropic), "Building effective agent automations", claude.dev blog (8 Oct 2026). https://claude.dev/blog/building-effective-agent-automations/

[12] D. Thomas (Notion), "How we built a personal data scientist for every Notion employee", Notion blog (8 Oct 2026). https://www.notion.com/blog/how-we-built-a-personal-data-scientist-for-every-notion-employee

[13] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala, A. Narayanan, "Towards a Science of AI Agent Reliability", ICML 2026; arXiv:2602.16666v3 (2 Jun 2026). https://arxiv.org/abs/2602.16666v3

[14] CMU 11-768, lecture 2 slides (tool use). https://www.cmu-agents.com/slides/lecture-02-tool-use.pdf

[15] CMU 11-768, lecture 5 slides (planning). https://www.cmu-agents.com/slides/lecture-05-planning.pdf

[16] CMU 11-768, lecture 13 slides (agent safety). https://www.cmu-agents.com/slides/lecture-13-agent-safety.pdf

[17] Stanford CS146S (Fall 2026), week 2 assignment. https://github.com/mihail911/modern-software-dev-assignments/blob/master/week2/assignment.md

[18] Stanford CS146S (Fall 2026), week 3 assignment. https://github.com/mihail911/modern-software-dev-assignments/blob/master/week3/assignment.md

[19] Northeastern, "Tools decide, or the model decides?" (week 3b). https://agenticllms.github.io/week3b-slides.html

[20] Northeastern, week 4a slides (workflows, routers, agents). https://agenticllms.github.io/week4a-slides.html

[21] Northeastern, DADA1 project specification, §9 "Getting started". https://github.com/agenticllms/lecture-files/blob/main/DADA1/DADA1_SPEC.md#9-getting-started

[22] Northeastern, research-project resources, "The rule". https://agenticllms.github.io/research.html

[23] University at Buffalo CSE 410/510 (Fall 2026), Labs 3–5 and Final Team Project. https://cse.buffalo.edu/courses/cse410/2026_fall/cse410ae/

[24] Universität Mannheim IE685, deck 4 (LLM agents and tool use). https://www.uni-mannheim.de/media/Einrichtungen/dws/Files_Teaching/Large_Language_Models_and_Agents/FSS2026/IE685_LA_04_LLMAgentsAndToolUse.pdf

[25] Universität Mannheim IE685, deck 8 (security and safety). https://www.uni-mannheim.de/media/Einrichtungen/dws/Files_Teaching/Large_Language_Models_and_Agents/FSS2026/IE685_LA_08_SecurityAndSafety.pdf

[26] J. A. Arroyo, *Learning to Build Agentic Systems: An instructional compilation guide built from public university courses* (October 2026). Document 1 of this set; indexes the courses cited here. File `guide/agentic_guide.pdf`.

[27] J. A. Arroyo, *Screening Tasks for Agentic Automation* (v0.9, October 2026). Document 3 of this set. File `task_screening/task_screening.pdf`.

[28] Universität Mannheim IE686, "Introduction to Student Projects" (IE686_LA_07). https://www.uni-mannheim.de/media/Einrichtungen/dws/Files_Teaching/Large_Language_Models_and_Agents/FSS2026/IE686_LA_07_IntroStudentProjects.pdf

[29] CU Boulder CSCI 7000-005, project proposal. https://danny.cs.colorado.edu/courses/csci7000-005_Sp26/projectProposal.html

[30] JHU EN.601.727, Assignment 3 README (§§1.2–3.3). https://github.com/machine-programming/assignment-3

[31] JHU EN.601.727, Assignment 2. https://github.com/machine-programming/assignment-2

[32] Stanford CS329A (Autumn 2025), Homework 3 notebook. https://github.com/stanford-cs329a/cs329a-homework3-fall2025-public/blob/main/homework3.ipynb

[33] Stanford CS329T (Fall 2025), HW1 handout. https://web.stanford.edu/class/cs329t/homeworks/CS329T_HW1_Fall2025.pdf

[34] Stanford CS329T, Lecture 3 slides (Agent GPA). https://web.stanford.edu/class/cs329t/slides/Lecture%203%20--%20CS%20329T%20Fall%202025.pdf

[35] UC Berkeley Agentic AI (Fall 2025), "LLM Agent Evaluations & Project Overview". https://rdi.berkeley.edu/agentic-ai/slides/LLM%20Agent%20Evaluations%20%26%20Project%20Overview.pdf

[36] UC Berkeley Agentic AI (Fall 2025), "Practical Lessons from Deploying Real-World AI Agents" (Sierra). https://rdi.berkeley.edu/agentic-ai/slides/Practical_Lessons_AI_Agents.pdf

[37] UC Berkeley Advanced LLM Agents (Spring 2025), Lab 2 instructions, "Recommended Implementation". https://drive.google.com/file/d/1_8ASqnVR99BTiKd-I7g8Ov7-UiXsJeHW/view

[38] UC Berkeley Agentic AI (Fall 2025), course page, "Grading" and "Project Timeline". https://rdi.berkeley.edu/agentic-ai/f25